

Claude Code *Karma*

Your Claude Code sessions deserve more than a terminal. Karma turns the raw data in `~/.claude/` into a visual story — sessions, timelines, costs, and live activity — **entirely on your machine**.

● Local-first

● No cloud · no accounts · no telemetry

● Apache-2.0

● CLI + Claude Desktop

01 What it is

Claude Code already saves everything locally — every session, every tool call, every token — as JSONL files buried under `~/.claude/`. You'll never read them by hand. **Karma reads those files and serves them through a local dashboard**, so the goldmine of data you're already generating becomes browsable, searchable, and beautiful.



Stays on your machine

The API reads your local files and indexes metadata in a local SQLite database. Nothing is ever uploaded.



Zero instrumentation

No SDK to wire in. It works with any session that writes to `~/ .claude/` — Claude Code CLI *and* Claude Desktop.

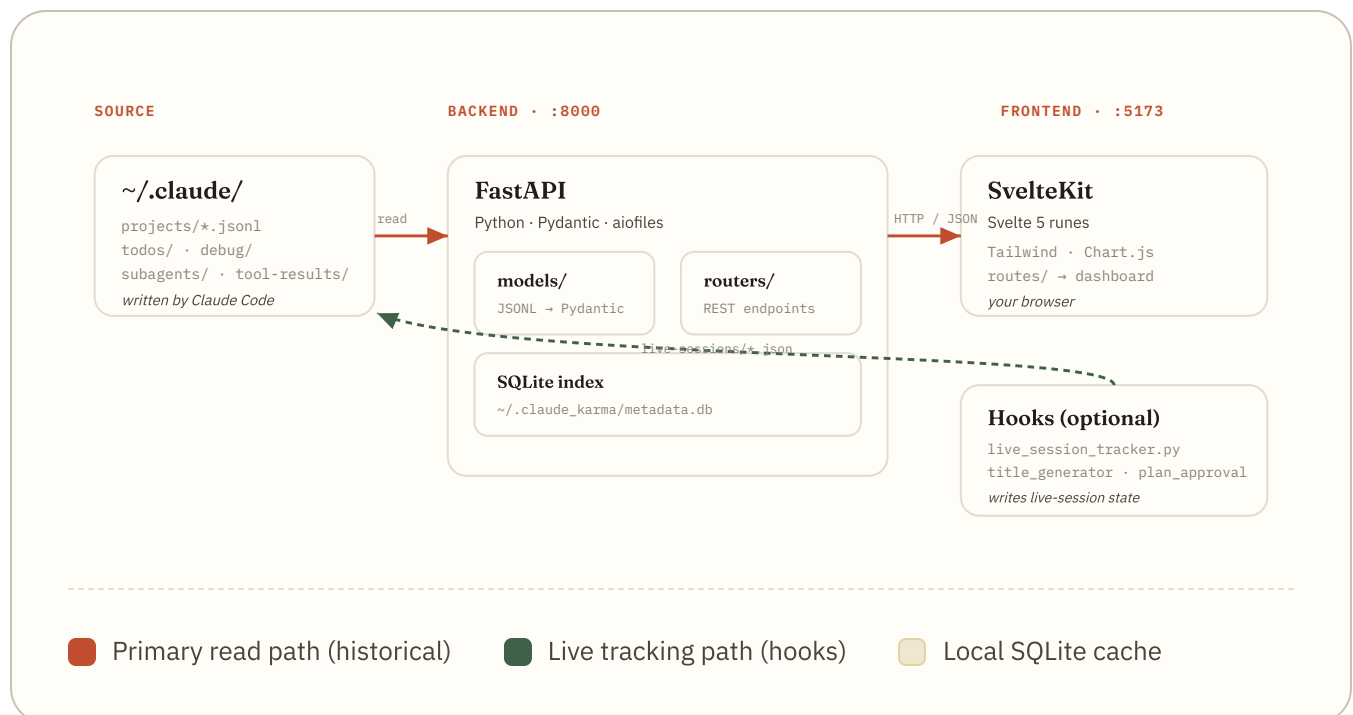


Live + historical

Optional hooks stream real-time session state, while the parser reconstructs the full history of everything that happened.

02 How the pieces fit

A three-tier pipeline: local files → a parsing API → a reactive dashboard. An optional hooks layer feeds live state in alongside the historical read.



03 What it does

Every surface is a different lens on the same local data. Filters persist in the URL so any view is shareable and bookmarkable.

Sessions

[/sessions](#)

Browse every session in one place. Full-text search across titles, prompts & slugs; filter by project, branch, status, and date. Live sessions surface at the top with real-time badges.

Session detail

[timeline](#)

Replay exactly what happened — every prompt, tool call, thinking block and response, chronologically. Dedicated tabs for Tasks, Files, Subagents, Skills, Commands & per-session Analytics.

Projects

[/projects](#)

Workspaces organized by git repository. Stat grids for tokens, cost, duration & cache hit-rate; active-branch filters; sessions grouped by recency. Aggregates across every checkout of a repo.

Analytics

[/analytics](#)

Token usage, cost, velocity trends, cache efficiency and coding rhythm across all projects — plus work-mode distribution (exploring vs. building vs. testing) over selectable time windows.

Agents & Subagents

[/agents](#)

Built-in, custom and plugin agents with run counts, token consumption, average duration and project breakdowns. Drill into any agent for its full session history.

Tools & Skills

[/tools](#)

Every built-in tool and MCP integration, grouped by server with call counts. Skills grouped by plugin with context split (main vs. subagent) and complete invocation history.

Plugins & Hooks

[/plugins](#)

Installed plugins with the agents, skills, commands, MCP tools & hooks they provide. Hooks are visualized by lifecycle phase, flagging which can block execution.

Ticket linking

[/tickets](#)

Attach sessions to Linear, Jira & GitHub issues/PRs. Link by URL, by `/link-ticket` command, or auto-detect from branch names. Read-only — caches title/status, never writes back.

04 How it does it

From raw JSONL on disk to a rendered chart — the data makes four hops, and never leaves the machine.

01

Read local storage

Claude Code persists each session as `{uuid}.jsonl` under a path-encoded project directory. Karma discovers projects, sessions, subagents, tool-results, todos and debug logs straight from `~/.claude/`.

02

Parse into typed models

The `models/` layer streams JSONL via `iter_messages()` (lazy loading for huge sessions) into frozen Pydantic models — `UserMessage`, `AssistantMessage`, summaries, agents, tool results. Compaction is detected from summary messages; related sessions are chained via leaf-uuid / slug matching.

03

Index & serve

Metadata is cached in a local SQLite db (`~/.claude_karma/metadata.db`) so listings stay fast, then exposed through FastAPI `routers/` — projects, sessions, timeline, tools, analytics, live-sessions, agents, skills, tickets and more.

04

Render in the browser

The SvelteKit frontend (Svelte 5 runes — `$state`, `$derived`, `$effect`) fetches those endpoints and renders cards, timelines and Chart.js visualizations. Filter state lives in URL search params.

Where the data lives

DATA	LOCATION ON DISK
Session transcript	~/.claude/projects/{encoded-path}/{uuid}.jsonl
Subagents	.../{uuid}/subagents/agent-*.jsonl
Tool results	.../{uuid}/tool-results/toolu_*.txt
Todos	~/.claude/todos/{uuid}*.json
Debug logs	~/.claude/debug/{uuid}.txt
Live session state	~/.claude_karma/live-sessions/{slug}.json
Karma metadata index	~/.claude_karma/metadata.db

Path encoding: a leading / becomes -, and every other / becomes - — so /Users/me/repo → -Users-me-repo.

Live session tracking

Opt-in Claude Code hooks write small JSON state files as a session moves through its lifecycle. The dashboard polls those files to show real-time status — no daemon, no socket, just the filesystem.

LIVE

actively running

WAITING

waiting for user input

STOPPED

agent finished, session open

STALE

05

Tech stack

Backend — port 8000

Python 3.9+

FastAPI

Pydantic 2.x

aiofiles

SQLite

pytest

ruff

■ Frontend — port 5173

SvelteKit 2

Svelte 5 (runes)

Tailwind CSS 4

Chart.js 4

bits-ui

lucide-svelte

TypeScript

06 Quick start

Two terminals. The API parses and serves; the frontend renders. Open the dashboard at `localhost:5173`.

```
# Terminal 1 — API
cd api
pip install -e ".[dev]" && pip install -r requirements.txt
uvicorn main:app --reload --port 8000

# Terminal 2 — Frontend
cd frontend
npm install && npm run dev

# → open http://localhost:5173
```